

Circularity Dashboard

A France–Brazil comparison of Extended Producer Responsibility

Code and data documentation

Arthur Briens
USP–PSL Institute for Circular Economy

August 28, 2026

Abstract

This document explains how the circularity dashboard is built: what each file does, why it is written the way it is, and which traps in the underlying data the code is deliberately defending against. It is written to be read by someone who has not seen the code before, and to be useful again in six months when the reasoning behind a particular line has been forgotten. Sections 2 and 3 are the ones to read first; the rest can be consulted as reference.

Contents

1	What this project is	3
1.1	The one idea behind the whole thing	3
2	Getting it running	3
2.1	Locally	3
2.2	Deployed	3
2.3	Where the data comes from	3
3	The mental model	4
3.1	Streamlit re-runs the whole script	4
3.2	The visual layer is three tiers that do not overlap	4
4	Repository map	5
5	The library layer	7
5.1	lib/data.py — getting the ADEME data in	7
5.2	lib/theme.py — one visual system	7
5.3	lib/market.py — monopoly, duopoly, oligopoly	8
5.4	lib/targets.py — the comparison schema	8
6	The pages	9
6.1	app.py — landing page	9
6.2	2_targets.py — France vs Brazil targets	9
6.3	3_EPR.py — EPR market structure	9
6.4	4_France_EPR.py — filière detail	10
7	The data files	10
7.1	REP.csv — put on market	10
7.2	traitements_REP.csv — treated waste	10
7.3	targets_fr_br.csv — the targets	11

8	Design rules that recur	11
9	Known limitations and next steps	11
A	Targets CSV schema	13
B	Troubleshooting	13

1 What this project is

The dashboard compares how France and Brazil regulate the circular economy. It has three substantive pages:

- **Targets** — what each country has committed to, in law, and how far it has got.
- **EPR market structure** — how concentrated the French compliance market is: monopoly, duopoly or oligopoly, filière by filière.
- **France, EPR filières** — tonnages placed on the market and treated, by filière, eco-organisme and region.

It is a Streamlit application written in Python, reading ADEME open data for the French side and a hand-assembled, sourced CSV for the targets comparison.

1.1 The one idea behind the whole thing

France and Brazil do not set targets on the same footing, and the easiest way to produce a misleading comparison is to put two percentages on one axis when their denominators differ. France measures WEEE collection as 65 % of a rolling three-year average of equipment placed on the market. Brazil measures it as 17 % of product weight against a fixed 2018 base year. Both are “a percentage of what was put on the market”, both can be plotted as a bar, and they are not the same quantity.

Almost every design decision in this codebase follows from taking that problem seriously. The data model carries a **basis** field (the denominator, written out in words) alongside every number, and the charts refuse to place two countries on a shared axis unless their bases actually match.

2 Getting it running

2.1 Locally

From the project root:

```
pip install -r requirements.txt
streamlit run output/dashboard/app.py
```

The app opens in a browser. The three pages appear in the left-hand navigation, which Streamlit builds automatically from the files in `output/dashboard/pages/`. The numeric prefix in `2_targets.py` sets the ordering; it is not part of the page name.

2.2 Deployed

The app is deployed on Streamlit Community Cloud from a private GitHub repository. The entry point is `output/dashboard/app.py`. Cloud rebuilds from a fresh clone on every push.

2.3 Where the data comes from

The three ADEME extracts are deliberately *not* committed to git; the targets CSV *is*. The line drawn is **third-party bulk extracts out, our own research in** — the ADEME files total roughly 4.5 MB and are redownloadable at any time, whereas `targets_fr_br.csv` is 84 KB of hand-assembled, individually sourced rows that the targets page needs in order to be reproducible from a clone.

Either way, each file is resolved in two steps:

1. `data/raw/<filename>` — the local working copy, used in development;

2. a URL held in `st.secrets["data"]` — used by the deployed app.

A fresh clone contains the targets CSV but none of the ADEME extracts, so those fall through to the secrets URL. Configure them on Streamlit Cloud under **Manage app** → **Settings** → **Secrets**:

```
[data]
msm_url = "https://.../REP.csv"
trt_url = "https://.../traitements_REP.csv"
ref_url = "https://.../Referentiels_MSM.xlsx"
targets_url = "https://.../targets_fr_br.csv" # optional: overrides the
                                              # committed copy
```

Locally the same block goes in `.streamlit/secrets.toml`, which is gitignored. `secrets.toml.example` at the repository root documents the shape and is safe to commit.

These URLs are credentials. A pre-signed S3 link or a share link with an embedded token grants read access to whoever holds it. They must never enter git, a log line, or an error message. `lib/data.py` contains a `_scrub()` helper whose only job is to strip URLs out of exception text before it is displayed, because an error message is the easy way to leak one.

3 The mental model

Two ideas explain most of the code. Neither is obvious from reading it.

3.1 Streamlit re-runs the whole script

Streamlit re-executes the *entire page script, top to bottom, on every interaction*. Move a slider and the file runs again from line 1. There is no callback system and no component tree that persists between interactions.

Three consequences follow, and they explain a lot:

- **Widgets are just values.** `dark = st.toggle(...)` returns a plain boolean for this run. There is no state plumbing.
- **Expensive work must be cached**, or a 2.6 MB CSV is re-read on every click. Hence `@st.cache_data` on every loader.
- **Declaration order is render order.** A control created after `st.columns()` appears *below* the charts it drives. This caused a real bug: the “filière in focus” selector rendered under the charts it controlled until it was moved above the column split.

3.2 The visual layer is three tiers that do not overlap

Styling a Streamlit app reaches different places depending on which lever you pull, and it is worth knowing which is which.

Tier	Reaches	Limitation
<code>.streamlit/config.toml</code>	Streamlit’s own widgets: sidebar, sliders, multiselects, buttons	Read once at server start. <i>Static</i> — cannot respond to anything in the script
Plotly template (<code>lib/theme.py</code>)	Axis colours, gridlines, fonts, hover styling, legend placement, the categorical colour order	Only applies to figures, and Streamlit will override parts of it unless told not to (see below)

Tier	Reaches	Limitation
Injected CSS (<code>theme.page_css</code>)	Metric cards, the hero number, section labels, chart card backgrounds, text colour in dark mode	Targets Streamlit's internal <code>data-testid</code> attributes, which can change between versions. The fragile tier — kept to a handful of rules

The seam. Because tier 1 is static and tiers 2–3 are dynamic, the in-app dark-mode toggle changes the charts and cards but *not* Streamlit's sidebar and widgets. That is a genuine half-measure. Making it complete would mean setting `base = "dark"` in `config.toml`, which is an app-wide, restart-level setting rather than a per-session toggle.

Two Plotly traps, both discovered the hard way.

1. `st.plotly_chart` defaults to `theme="streamlit"`, which overrides your registered template. Every call in this codebase passes `theme=None`.
2. Even then, Streamlit merges `config.toml`'s colours over the figure's *template* values, because template values count as defaults. Only an explicit value on the figure's own layout wins that merge. So every chart helper sets `paper_bgcolor` and `plot_bgcolor` explicitly. Without this, dark mode rendered every plot area white under a dark page.

4 Repository map

Path	Lines	Role
output/dashboard/app.py	95	Landing page. Project description and links to the three pages
lib/__init__.py	1	Makes <code>lib</code> a package
lib/data.py	514	Source resolution, ADEME loaders, French transforms
lib/market.py	314	Market-structure measures (monopoly/oligopoly)
lib/targets.py	549	Targets schema, placeholder set, loader, pairing
lib/theme.py	206	Palette, Plotly templates, page CSS
pages/2_targets.py	492	France vs Brazil target comparison
pages/3_EPR.py	568	EPR market structure
pages/4_France_EPR.py	446	France filière detail
data/raw/REP.csv	1.8 MB	Put-on-market declarations, 2017–2024
data/raw/traitements_REP.csv	2.6 MB	Treated-waste declarations
data/raw/Referentiels_MSM.xlsx	76 KB	Actor and filière lookups
data/raw/targets_fr_br.csv	81 KB	194 researched targets
docs/targets_csv_schema.md	—	Specification for the targets CSV
requirements.txt	—	pandas, openpyxl, streamlit, plotly
secrets.toml.example	—	Template for the data-source URLs
.streamlit/config.toml	—	Streamlit theme (accent colour, fonts)
code/*.py	—	Earlier exploratory analysis, not used by the app

The split is deliberate: `lib/` contains everything that touches pandas and nothing that touches presentation; `pages/` contains composition and almost no data logic. A practical benefit is that you can debug a chart without launching Streamlit at all:

```
from lib import data as D, market as M
panel = M.actor_panel(D.load_msm(), D.load_actors())
M.structure(panel).query("filieres == 'EEE'")
```

5 The library layer

5.1 lib/data.py — getting the ADEME data in

Three responsibilities: find the files, load them, reshape them.

Finding the files

A small `Source` dataclass names each required file, its expected local filename, and its key in the secrets table. `_open(key)` returns either a `Path` or an in-memory buffer, and pandas accepts both, so the loaders do not care which they got.

`require_data()` is called at the top of every page that needs data. It does two things, both about failing legibly:

1. checks that each source is configured, and if not, prints a message naming the missing file and both ways to fix it;
2. *primes* the loaders inside a `try`, so that a download that times out or returns 403 is reported here rather than surfacing deep inside a chart.

This matters because Streamlit Community Cloud **redacts exception text** to avoid leaking data. An unguarded pandas error reaches the user as “the original error message is redacted” with no indication of which file is missing — which is exactly the failure this project hit in deployment.

`require_data()` is deliberately *not* cached: `st.stop()` raises a control-flow exception, and caching a function that raises it would poison the cache entry.

Path resolution

```
ROOT = Path(__file__).resolve().parents[3] # lib -> dashboard -> output -> repo
RAW = ROOT / "data" / "raw"
```

Paths resolve from the file, not the working directory. A relative `"data/raw/REP.csv"` only works when the app happens to be launched from the project root, which is not guaranteed on a host.

Reshaping, and the traps it defends against

The transform functions each return a tidy frame shaped for exactly one chart. Three defensive details are worth knowing, because each was a real bug:

- `type_ton` is **null for most filières**. Any `groupby` or `merge` on it silently drops EEE, TLC and EA. It is filled with "N/A" before it is ever used as a key.
- `pays_site_trt` is **null for all EMPAP rows**. Treating null as “not France” rendered EMPAP as 100 % exported, which is nonsense. The field is now three-state: France, abroad, or *not declared*.
- `reg_site_trt` uses **pre-2016 region codes** — 22 régions, not the current 13. `LEGACY_REGION` maps them across.

5.2 lib/theme.py — one visual system

Pure parameters, no data knowledge. Two palette dictionaries (light and dark), a Plotly template factory, and a CSS string builder.

The categorical colour order is validated for colour-vision deficiency in both modes. **The order is the safety mechanism, not decoration.** An attempt to re-order the palette so that two particular hues were not adjacent failed validation badly in dark mode, and was reverted.

Three colour roles, and mixing them up is the most common way a chart goes wrong:

- **Categorical** — identity, no order. Countries, eco-organismes.
- **Ordinal** — ranked categories. The waste hierarchy; the monopoly-to-oligopoly scale. Darkest is “most concentrated”.
- **Sequential** — continuous magnitude. Heatmaps only.

Status colours (good/warning/critical) are reserved and never reused as a series colour. This matters in practice: Plotly assigns marker colour from the colourway if you do not specify one, so target markers silently came out green — reading as “good” next to a chart about missing targets — until they were coloured explicitly.

5.3 lib/market.py — monopoly, duopoly, oligopoly

A REP filière is a regulated market: producers must join an eco-organisme or run an approved individual system, so the declarants in `REP.csv` are that market’s suppliers of compliance. The standard industrial-organisation toolkit therefore applies.

Two classifications, deliberately.

- **Structure** counts declarants: monopoly (1), duopoly (2), tight oligopoly (3–4), broad oligopoly (5+).
- **Effective** looks at what the leader actually holds: sole declarant, effective monopoly ($\geq 95\%$), dominant leader ($\geq 50\%$), or contested.

They disagree, and the disagreement is the finding. PCHIM has three accredited declarants — a “tight oligopoly” by any count — and a leader holding essentially the whole market. Both thresholds are named constants so they are easy to argue with.

The coverage trap. Filières enter the SYDEREP reporting system at different dates: PNEU from 2017, EEE from 2019, ABJ from 2023. A filière with zero declarants in a year is *not* a market with no suppliers — it is a market not yet reporting. Every function returns only reported filière-years, and the charts render the rest as blank. Reading a blank as a zero is the easiest mistake to make with this dataset.

Entry and exit are measured inside each filière’s *own* reporting window. A body present in the filière’s first reported year is the incumbent at the point the record begins, not an entrant. Measuring against the dataset’s overall window would invent a wave of entries in whichever year each filière joined the system.

5.4 lib/targets.py — the comparison schema

The point of this module is the schema, not the numbers.

Each row carries *both* a verbatim **metric** (as the instrument words it) and a normalised **metric_family** (how we choose to group it), plus an explicit **basis** string and a **comparable** flag. **metric_family** is our editorial judgement, not the law’s — it is the honest place for the comparison to be contestable, and it lives in a module-level list so it is easy to change.

The pairing rule. `paired()` matches on **sector** alone, deliberately *not* on (sector, metric_family). France measures packaging as a recycling rate and Brazil as reverse-logistics recovery, so keying on both fields matches nothing and the chart comes up empty — hiding the very comparison the page exists for. Instead:

1. if both countries share a metric family for that sector, restrict to it and take the furthest horizon within it;
2. otherwise fall back to the furthest horizon overall and set `cross_family`, so the chart can warn that the two bars measure different quantities. That is a stronger warning than a basis mismatch.

A boolean trap worth remembering. `comparable` was originally parsed with `Series.astype(bool)`. If a single row breaks the strict TRUE/FALSE pattern — a `no`, an `n/a`, a blank — pandas reads the column as text, and `astype(bool)` makes *every non-empty string True*, including the literal `"false"`. Every pair would have been silently marked as sharing a denominator, which is precisely the error the page exists to prevent. It now parses text explicitly and treats anything unrecognised as `False`, the safe direction.

6 The pages

6.1 `app.py` — landing page

Project description, institutional context, and `st.page_link` buttons to the three pages. No data access, so it always loads even when the data sources are misconfigured.

6.2 `2_targets.py` — France vs Brazil targets

Two sections, mirroring how the two countries legislate.

Section 1, government targets. France: EU directives transposed nationally, plus the loi AGECE, plus regional plans. Brazil: the PNRS framing law, the Planares national plan, and state plans. Two charts:

- *When each commitment bites* — a dumbbell from baseline year to target year. This is the only chart on the page with **no** comparability caveat, because time is genuinely shared even when the metrics are not. It surfaces something the numbers cannot: France’s commitments cluster at 2025–2035, Brazil’s stretch to 2040.
- *Distance still to travel* — bullet bars where each row is scored against *its own* target only, so no cross-country axis comparison is implied.

Section 2, sector-specific targets. France’s per-filière obligations are market-wide and mandatory; Brazil’s *acordos setoriais* bind signatory firms. That difference is structural and survives any amount of data cleaning.

Markers used in the paired chart:

- $\neq$ — the two countries measure this sector on different denominators;
- $\triangle$ — they do not even measure the same quantity, so there is no shared metric family to pair on. Read those as “both regulate this sector”, never as “one is ahead”.

Provenance is a first-class feature. A meter at the top reports what share of rows carry `status = verified`. The `status` field has three values: `placeholder` (invented, layout only), `sourced` (real, cited, but not read in the primary legal text) and `verified` (read in the instrument’s own text). Only the last counts. The middle value exists because “placeholder” would misdescribe a row that is real but second-hand, and “verified” would overclaim it.

6.3 `3_EPR.py` — EPR market structure

Reads the put-on-market declarations as an industrial-organisation problem: how much choice does a producer actually have?

- **A** — headline counts: single-declarant filières, effective monopolies, median leader share, genuinely contested filières.
- **B** — structure map, filière $\times$ year, on the ordinal ramp with monopoly darkest. EEE’s staircase reads at a glance: monopoly (2019) $\rightarrow$ duopoly (2020) $\rightarrow$ tight oligopoly (2021) $\rightarrow$ broad oligopoly (2022).
- **C** — the chart that does the real work: each filière split into leader / 2nd / 3rd / all others, sorted by leader share, with the declarant count in brackets. This is where nominal and effective structure visibly diverge.
- **D** — a transition log and an entry/exit dot chart.
- **E** — one filière in detail: share bands over time, and a timeline of which declarants were active when.
- **F** — operators spanning several filières. Single-filière views hide the bodies accredited across many at once.
- **G** — route to market, from the `statut_prod` field.

6.4 4_France_EPR.py — filière detail

Reading order: status $\rightarrow$ trajectory $\rightarrow$ mechanism $\rightarrow$ territory. Headline tiles, a gap-to-target bullet chart, indexed trajectories as small multiples, treatment composition on the waste-hierarchy ramp, eco-organisme shares, domestic versus exported treatment, and a schematic tile map by region.

Two honest limits are stated on the page itself. Every rate here is `treated mass / tonnage put on market`, a proxy, because the separate collection dataset is not among the extracts. And the target values in the gap chart are placeholders held in `lib/data.PLACEHOLDER_TARGETS`.

Why the map is a tile grid, not a choropleth. Every région gets equal area, so colour is not biased by territory size and Île-de-France does not vanish beside Nouvelle-Aquitaine. The value is printed in each tile, so colour never carries the information alone. It also handles the overseas régions without the usual inset awkwardness — relevant, because their collection rates are markedly lower.

7 The data files

7.1 REP.csv — put on market

29,485 rows, 2017–2024, 18 filières. One row per declaring actor per filière per year. Key columns: `annee`, `filiere`, `acteur`, `statut_prod`, `quantite`, `tonnage`.

`acteur` is a code such as FR000017; `Referentiels_MSM.xlsx` maps it to a company name. The declarants are a mix of eco-organismes and individual systems, and both are kept: an individual system is a genuine alternative to joining a collective body, so excluding them would overstate concentration.

7.2 traitements_REP.csv — treated waste

34,971 rows. **Semicolon-separated**, unlike `REP.csv`. Adds `typ_trt` (73 treatment codes, including legacy variants), `reg_site_trt`, `dep_site_trt` and `pays_site_trt`.

Two things to hold on to. First, the location fields refer to where waste was **treated**, not collected — a région with high treated tonnage has treatment plants, not necessarily good collection. Second, the 73 treatment codes are mapped to four waste-hierarchy buckets by ordered prefix rules in `lib/data._RULES`; anything unmatched shows as *Unclassified* rather than being folded into a bucket. If that share is large, fix the rules before trusting the composition chart.

TLC’s 2024 treatment is entirely `TRI_INCONNUE` — sorted, destination unknown. It renders as a full grey Unclassified bar. That is the data telling you something real about textile traceability, not a mapping failure.

7.3 `targets_fr_br.csv` — the targets

194 rows, 19 columns, every row carrying a source URL. 79 government-level and 115 sector-level; 66 % marked **verified**. This is the one data file committed to the repository, for the reason given in Section 2.3. See Appendix A for the column specification and `docs/targets_csv_schema.md` for the full version.

Notable content: for France, the EU directives (Waste Framework, Landfill, Packaging including all six per-material targets, Single-Use Plastics, WEEE, Batteries, ELV, PPWR), the loi AGECE, décret 3R, and the REP *cahiers des charges* for nine filières. For Brazil, Planares Metas 1–9 with their 2024/2028/2032/2036/2040 trajectories, the staggered *lixão* deadlines, and the reverse-logistics instruments for plastics, EEE, tyres, lubricants, lamps, medicines and agro-chemicals.

Three caveats carried in the file itself.

- The Planares metas were transcribed from documents reproducing the decree’s annex, not read in the annex itself. The 2040 endpoints are corroborated; the intermediate years are the weakest cells.
- Brazil’s two national data sources disagree systematically. SINISA puts recovery at 1.82 % (2023); ABREMA at 8.7 % (2024). The gap is structural — ABREMA imputes the tonnage handled by roughly 700,000 informal *catadores*, which SINISA cannot see. Both are carried as separate rows rather than picking one.
- **Not one pair shares a denominator.** `comparable` is `FALSE` on all 194 rows. That is a finding, not an omission.

8 Design rules that recur

Five rules show up across every page. They are worth stating once.

1. **Blank is not zero.** Missing data renders as an empty cell, never as a zero bar. Both heatmaps had to be fixed for this: one printed the literal `NaN`, the other the literal `null`.
2. **Name the denominator.** Every rate carries a `basis` string in words. Two percentages may only share an axis when their bases match.
3. **Say what is missing.** When a filière drops out of a ranking for want of data, the page names it. A row that silently vanishes reads as “not applicable” rather than “no data”.
4. **Distinguish absence from zero in the data model too.** Brazil has *no* battery collection target; the CSV records that explicitly rather than leaving the cell blank and letting the reader infer.
5. **Sub-1 % is not 0 %.** An operator present with 0.3 % of a filière is present. Those cells read `<1%`.

9 Known limitations and next steps

- **No collection dataset.** Every rate on the France page is `treated / put on market`, a proxy. ADEME publishes *REP — Tonnes issues des lieux de collecte* separately; adding it would make the rates real and let the gap-to-target chart use published rates rather than a computed proxy.

- **The gap-to-target values are placeholders.** They live in `lib/data.PLACEHOLDER_TARGETS` and should be replaced with the published figures from the ADEME filière fact sheets — most of which are now in `targets_fr_br.csv` and could be read from there instead.
- **Nothing is municipal.** Per the project’s granularity rule, the finest available level is `dep_site_trt`, the department of the *treatment site*. Municipal analysis would need a different source.
- **Dark mode is partial.** Charts and cards follow the toggle; Streamlit’s own widgets do not (Section 3).
- **Brazil’s side has no equivalent to the French filière data.** The targets page compares commitments; there is no Brazilian counterpart to `REP.csv` in the project yet, so the market-structure analysis is France-only.
- **Pre-signed URLs expire.** If the deployed app works today and returns 403 next week, that is the first thing to check.

A Targets CSV schema

Plain CSV, comma-separated, UTF-8, one header row, 19 columns in any order. Save from Excel as “CSV UTF-8 (Comma delimited)” — a French or Portuguese locale defaults to semicolons, which breaks the read.

#	Column	Required	Notes
1	country	yes	France or Brazil exactly. Any other value rejects the whole file
2	level	yes	government or sector , lowercase. Chooses the page section
3	tier	no	e.g. EU , National , Federal , Sector agreement
4	sector	no	The pairing key . Must match exactly between the two countries
5	metric_family	no	One of the six normalised families
6	metric	no	The metric as the instrument words it
7	basis	no	The denominator, in words . Load-bearing
8	unit	for charts	%, Mt, kg/cap, count . Only % rows reach the bar charts
9–14	the six numeric fields	no	baseline_value/year , latest_value/year , target_value/year
15	instrument	no	Cite the instrument, not a summary
16	comparable	no	TRUE/FALSE . Anything unrecognised is treated as FALSE
17	status	yes	placeholder , sourced or verified
18	note	no	Caveats, disagreements between sources
19	source_url	no	Where the figure came from

The six metric families are: Recycling rate; Separate collection rate; Landfill diversion; Collection coverage; Reuse & preparation for reuse; Reverse-logistics recovery. A value outside this list is filtered out of the page and effectively invisible.

Rejection vs. silent coercion. The loader rejects the whole file (falling back to placeholders, with the reason shown in red) if a column is missing or **country/level** contains an unexpected value. It does *not* reject, but silently blanks, a non-numeric value in a numeric column — so 50%, ~50 and n/a all quietly become empty. Write bare numbers.

B Troubleshooting

Symptom	Cause and fix
<code>FileNotFoundError</code> , message redacted, on the deployed app	The data files are not in the deployment. Either commit <code>data/</code> or configure the secrets URLs (Section 2.3). The pages now catch this and print which file is missing
Targets page shows the placeholder banner	No <code>targets_fr_br.csv</code> found locally and no <code>targets_url</code> in secrets

Symptom	Cause and fix
Red “found but rejected” banner	A column is missing, or <code>country/level</code> has an unexpected value. The message names the problem
Charts render white in dark mode	A <code>st.plotly_chart</code> call is missing <code>theme=None</code> , or the figure does not set <code>paper_bgcolor</code> explicitly
A control appears below the charts it drives	It was declared after <code>st.columns()</code> . Move it above the column split
Deployed app returns 403 on data	A pre-signed URL has expired. Regenerate it and update the secrets
A filière is missing from a chart	Check the caption underneath — pages name what they dropped and why